

RuleLab - Project Documentation

Firewall-rule safety simulator on the NSL-KDD dataset

Generated 2026-10-09. Every figure below was read from files in this repository on that date; nothing is estimated.

1. Problem

A firewall rule that blocks attacks also blocks legitimate traffic. Before deploying, a network defender needs to know: how much labeled-normal traffic would this rule take down, how does that change on data the rule was not tuned on, and what happens when an attack family appears that the rule never saw. RuleLab answers all three offline, on a public dataset, with numbers that are checked by automated tests.

2. User

A network or security administrator evaluating firewall rules. This is the persona stated in the explanation system prompt (llm.py, SYSTEM_PROMPT: "You explain firewall-rule metrics to a network or security administrator"). The UI assumes no machine-learning background: rules are readable conditions, collateral is a percentage with a SAFE/CAUTION/UNSAFE pill.

3. USP 1 - Live collateral meter

The Builder tab assembles a rule (3 categorical selectors plus up to 3 numeric conditions, values taken from fit-split quantiles) and evaluates it against Validation and KDDTest+ in the same render: attacks blocked, normal blocked (collateral), precision, verdict, cap drift, a cap-usage gauge, and a per-family table that prints $n < 30$ instead of a percentage for families under 30 rows. Evaluation runs off precomputed condition masks, so a slider move updates both datasets instantly.

How it is proven (tests/smoke_ui.py, 19/19 PASS, 68.9 s wall)

- p05_c1_both_columns: both dataset columns render attacks-blocked and normal-blocked metrics - PASS.
- p05_c2_slider_move_changes_both: moving one slider changes attacks blocked AND normal blocked in the same render - PASS.
- p05_c3_median_under_100ms: 50 evaluations of a 3-condition rule over all 37,792 validation rows: median 34.12 ms, p95 43.20 ms (target < 100 ms) - PASS.
- p05_c4_preset_*: the "flag = S0" preset shows exactly the B0 numbers recorded in results/eval.json on both splits - PASS.
- p05_c5_delta_appears: delta arrows appear after a change - PASS.
- p05_c6_n_lt_30_rows: families with fewer than 30 rows show $n < 30$ and never a percentage - PASS.

4. USP 2 - Zero-day rule drill

The drill removes every row of one attack family from the fit split, re-mines rules on the remainder, and compares recall of the seen rule set against the unseen (family-excluded) rule set on validation and test. It runs for all 10 eligible families ($n_{\text{train}} \geq 100$) at all 4 caps and writes results/drill.json.

How it is proven

- All 40 family x cap entries in results/drill.json pass their three assertions: assert_0_f_rows_in_unseen_fit, normal_rows_untouched, identities_ok (0 failures, verified).
- p06_c6_rerun_matches_json - PASS: a live re-run in the UI reproduces drill.json exactly for neptune at 1% cap (seen val recall 0.834115, unseen 0.010919, live runtime 1.9 s).
- p06_c6_two_families_render and p06_c6_rerun_no_exception - PASS.
- Honest outcome, reported as-is: at the default 1% cap 0 of 10 reliable families keep >= 50% unseen validation recall; median seen-unseen gap 0.00 pp. Only neptune separates (seen 83.41% vs unseen 1.09%, gap 82.32 pp).

5. Dataset

NSL-KDD text files in data/, loaded by data.py with hard validation: every line must have exactly 43 fields, no header, non-numeric labels, numeric difficulty; labels are lowercased, trailing dots stripped; the loader aborts on any violation.

file	rows	fields	labels	largest label	nulls	difficulty
KDDTrain+.txt	125,973	43	23	normal 67,343	0	0-21
KDDTest+.txt	22,544	43	38	normal 9,711	0	0-21

KDDTest+ labels absent from train: 17 (mscan, apache2, processtable, ...)

41 features are used (38 numeric + 3 categorical: protocol_type, service, flag); label and difficulty are excluded. Train is split 70/30 stratified by label with seed 42: fit 88,181 rows (41,041 attacks), validation 37,792 rows (17,589 attacks). The fit-index sha256 is b7d540719ba6389d0742072aebd79331f6fbbf916864e0efcc17f06ab6ccc50d. KDDTest+ (22,544 rows) is held out entirely.

6. Architecture (Mermaid)

The block below is the Mermaid source that also appears in README.md; it is reproduced as text because this PDF is generated without a diagram renderer.

```

flowchart TD
    subgraph raw["Raw data (read-only)"]
        T["data/KDDTrain+.txt<br/>125,973 x 43"]
        S["data/KDDTest+.txt<br/>22,544 x 43"]
    end
    end
    T --> D["data.py<br/>field/header/label validation<br/>stratified split seed 42"]
    D --> FIT["fit 88,181"]
    D --> VAL["validation 37,792"]
    FIT --> M["miner.py<br/>greedy: max 5 rules x 3 conditions,<br/>min 20 attacks, hard colla"]
    teral cap"]
    FIT --> C["app.py precompute<br/>condition masks x 3 splits"]
    R["rules.py<br/>Condition, metrics, verdict"] --- M
    R --- E["evaluate.py<br/>--eval / --decide / --drill"]
    VAL --> E
    S --> E
    E --> EJ["results/eval.json"]
    E --> DJ["results/decisions.json<br/>(test never loaded)"]
    E --> DRJ["results/drill.json<br/>10 families x 4 caps"]
    FIT --> APP["app.py - Streamlit<br/>Builder | Miner | Drill | Evaluation"]
    VAL --> APP
    S --> APP
    C --> APP
    EJ --> APP
    DRJ --> APP
    APP -->|Explain this rule"| L["llm.py<br/>grounding check + template fallback"]
    L --> G1["gemini-3.5-flash-lite"]
    G1 -->|"429/503/timeout/error"| G2["gemini-3.1-flash-lite"]
    G1 --> GND{"grounding_ok?"}
    G2 --> GND
    GND -->|yes| UI["explanation + source badge"]
    GND -->|no / no key / daily limit| UI2["deterministic template"]
    TST["tests/: 28 unittest + smoke_ui"] -.-> APP

```

7. Data flow

1. data.py reads a raw file only if all validation gates pass (43 fields per line, numeric first field, non-numeric label column, numeric difficulty column), then normalizes labels and derives is_attack.
2. split_fit_val stratifies by label with seed 42 and tags each split (fit.attrs["split"] = "fit").
3. miner.py::mine accepts fit rows only (a hard assertion rejects validation or test rows) and greedily grows up to 5 rules of up to 3 conditions with score = TP - lambda*FP, stopping before cumulative fit false positives would exceed the cap.
4. evaluate.py scores B0 (fixed flag == S0), B1 (one rule, one condition) and B2 (full miner) at caps 0.1/0.5/1.0/2.0% on fit, validation and test; the five verification checks V1-V5 must pass or the run exits non-zero; results/eval.json is written. --decide patches the test loader to raise so KDDTest+ can never influence the decision, and writes results/decisions.json. --drill writes results/drill.json.
5. app.py reads the JSON files (or prints the exact command that creates them) and computes Builder metrics from cached condition masks; mining happens only on an explicit button press, never on page load.

8. AI flow (optional)

1. A JSON context of the numbers already on screen is built (rule text plus recall, collateral, precision, verdict per dataset).

2. explain() checks the cache first (no request), then the key: missing GEMINI_API_KEY returns the deterministic template with reason "missing API key".
3. A daily budget of 450 calls is enforced through results/llm_usage.json; at the limit the template is used with reason "daily limit reached".
4. Otherwise a POST goes to generativelanguage.googleapis.com/v1beta/models/{model}:generateContent with the key in the x-goog-api-key header only, temperature 0.2, maxOutputTokens 300, 30 s timeout, at least 4.5 s between calls.
5. The primary model is tried first (one retry on timeout/429/503), then the fallback model; every number in the reply must appear in the context (grounding_ok) or the template is used with reason "grounding failed".
6. The UI shows a source badge: primary, backup, or template. Nothing is called when the app loads (0 HTTP calls, asserted in smoke_ui.py).

Gemini model IDs (pinned in config.py)

```
primary    GEMINI_MODEL          = gemini-3.5-flash-lite
fallback   GEMINI_FALLBACK_MODEL = gemini-3.1-flash-lite
```

9. Measured metrics (from results/eval.json, run 2026-10-09T13:34:53.897896+00:00)

Verification checks - all true

```
V1 5 fixed rules match df.query() on fit
V2 12 entries x 3 splits consistent
V3 5 rules identical across 2 runs at cap 0.01
V4 B2 fit collateral <= cap at every cap
V5 mine() rejected val and test rows
split sizes: fit 88,181 / val 37,792 / test 22,544
runtime_s 17.29  max_rules 5  max_conds 3  min_tp 20  fallback_max_conds null
```

All 12 entries (collateral in %, recalls as fractions)

cap	meth	rules	val_col%	val_rec	val_macro	test_col%	test_rec
0.1%	B0	1	0.4900	0.5889	0.0809	0.0000	0.1569
0.1%	B1	1	0.0346	0.5836	0.0970	0.0309	0.1318
0.1%	B2	5	0.0000	0.5692	0.0787	0.0000	0.1253
0.5%	B0	1	0.4900	0.5889	0.0809	0.0000	0.1569
0.5%	B1	1	0.0346	0.5836	0.0970	0.0309	0.1318
0.5%	B2	5	0.0000	0.5903	0.0824	0.0000	0.1331
1.0%	B0	1	0.4900	0.5889	0.0809	0.0000	0.1569
1.0%	B1	1	0.0346	0.5836	0.0970	0.0309	0.1318
1.0%	B2	5	0.0000	0.5903	0.0824	0.0000	0.1331
2.0%	B0	1	0.4900	0.5889	0.0809	0.0000	0.1569
2.0%	B1	1	0.0346	0.5836	0.0970	0.0309	0.1318
2.0%	B2	5	0.0000	0.5940	0.0977	0.0206	0.1385

Default-cap B2 rule set (cap 1.0%), rule_text copied from the file:

```
dst_host_srv_serror_rate >= 1 AND flag == S0 AND dst_host_diff_srv_rate >= 0.02 ;
service == Z39_50 ; service == uucp ; service == bgp ; service == uucp_path
```

From results/decisions.json (validation only; test never loaded)

DRIFT_TOL = 0.0025 (0.25 pp): max observed val-minus-fit collateral drift was 0.0000 pp across the 4 caps, so the 0.25 pp floor applied. RECOMMENDED_CAP = 0.02, reason "highest val macro-recall 0.0977 among 4 SAFE caps"; per-cap validation macro-recall 0.0787 / 0.0824 / 0.0824 / 0.0977, all verdict SAFE.

From results/drill.json (runtime_s 48.27, 10 families x 4 caps)

"10 eligible families at cap 1.0%: 0 of 10 reliable families (n_val>=30) keep unseen validation recall >= 50%; median seen-unseen validation recall gap 0.00 pp."

Test and performance measurements

- python -m unittest discover tests: Ran 28 tests, OK (0.076 s).
- tests/smoke_ui.py: 19/19 checks PASS (68.9 s wall).
- Cold-start runtimes on this machine: --eval 17.29 s (in-file), --decide 14.8 s (wall), --drill 48.27 s (in-file), data.py schema report 6.8 s wall (KDDTrain+ load 3.69 s, KDDTest+ load 0.61 s).
- Builder benchmark: median 34.12 ms, p95 43.20 ms for a 3-condition rule over all validation rows (50 runs).

10. Limitations (honest)

1. NSL-KDD is a 1999-era simulation, not real network traffic; the app shows this as a permanent banner.
2. Collateral counts labeled-normal rows only; it is not a business-cost estimate and knows nothing about traffic volume or importance.
3. Rule sets are conservative: the best SAFE validation macro-recall across caps is 0.0977, i.e. most families are barely covered.
4. Generalization gap: B2 validation recall 0.5692-0.5940 versus test recall 0.1253-0.1385 - KDDTest+ differs from the train split.
5. The drill shows the miner does not learn most families: 0/10 reach 50% unseen validation recall at the 1% cap; only neptune shows a real seen/unseen gap (82.32 pp).
6. 17 attack families appear only in KDDTest+ and can never be trained on; families under 30 rows are shown as n<30 instead of a percentage.
7. The miner is greedy (not optimal), capped at 5 rules x 3 conditions, single seed 42, no cross-validation; the difficulty column is validated but never used as a feature.
8. The collateral cap is enforced on the fit split only; validation/test collateral is measured, not guaranteed (e.g. 0.0206% on test at cap 2%).
9. Verdict thresholds (UNSAFE > 2%, CAUTION > 0.5% or drift > 0.25 pp, LOW EVIDENCE < 20 attacks) are fixed product rules documented in DECISIONS.md, not learned from data.
10. The LLM layer is optional: 450 calls/day, a heuristic grounding check, template fallback; no metric in the app depends on it; .env is auto-loaded via python-dotenv.
11. This is an offline simulator: it does not read live traffic, talk to firewalls, or deploy rules.
12. Performance figures are from single runs on one machine and will vary.

11. Run instructions

Verified on Windows PowerShell with the repository venv (Python 3.13.5). Every command below was executed successfully on this machine; the venv creation line was verified against a scratch path.

```
python -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt

.\.venv\Scripts\python.exe data.py           # schema report (6.8 s)
.\.venv\Scripts\python.exe evaluate.py --eval # results/eval.json (17.3 s)
.\.venv\Scripts\python.exe evaluate.py --decide # results/decisions.json (14.8 s)
.\.venv\Scripts\python.exe evaluate.py --drill # results/drill.json (48.3 s)

.\.venv\Scripts\python.exe -m unittest discover tests # 28 tests
.\.venv\Scripts\python.exe tests\smoke_ui.py          # 19 checks

.\.venv\Scripts\python.exe -m streamlit run app.py      # UI at http://localhost:8501
```

The Evaluation and Drill tabs need their JSON files; if a file is missing the tab prints the exact command that creates it. Explanations work with no API key (deterministic template text); the key is only needed for model output.

12. Environment variables

Read by config.py with os.getenv; names are listed in .env.example.

```
GEMINI_API_KEY      default: empty   missing/empty -> template, no request
GEMINI_MODEL        default: gemini-3.5-flash-lite  primary model
GEMINI_FALLBACK_MODEL default: gemini-3.1-flash-lite  tried after failure
```

A .env file is auto-loaded at startup via python-dotenv (1.2.4); variables already set in the shell take precedence.

13. Files

```
config.py      constants: split, caps, thresholds, model IDs, columns
data.py        loader + schema report + stratified split
rules.py       Condition, mask evaluation, metrics, verdict
miner.py       greedy rule miner (fit split only, hard cap)
evaluate.py    --eval / --decide / --drill -> results/*.json
llm.py         Gemini calls, grounding check, template fallback
app.py         Streamlit UI (Builder, Miner, Drill, Evaluation)
.streamlit/config.toml  dark theme
requirements.txt  numpy, pandas, requests, streamlit (pinned)
.env.example     environment variable names
DECISIONS.md     provenance of every threshold value
tests/           test_rules.py (18), test_llm.py (10),
                  smoke_ui.py (19 checks)
data/            KDDTrain+.txt, KDDTest+.txt (read-only)
results/         eval.json, decisions.json, drill.json,
                  llm_usage.json (after first model call)
README.md        project readme
PROJECT_DOCUMENTATION.pdf  this document
```